

LEAN: Linear Algebra and Relations

David Bataresh, Hemkesh Bandi, Eric Klavins, Haoming Ning

Code
Repository



What is LEAN?

LEAN is built on the calculus of construction with inductive types by Leonardo de Moura during his time at Microsoft Research (MSR) and continued while at Amazon (AWS) founded under the goal of automated theorem proving.

Making it:

A Proof Assistant



A Functional Programming Language



Notable Work

Mathlib – The Mathematical Library for LEAN. Big initiative to house the world of mathematics in LEAN.

- +216k theorems & 105K definitions

Liquid Tensor – Peter Scholze was worried his work was wrong. Asked the LEAN community to formalize it.

- 60,000 lines of computer verifiable code

Alpha Proof – DeepMind effort to have LLM automated theorem proving

- Silver medal on Math Olympiad

- Evolving sector with DeepSeek Prover V2 and even more recently Kimina Prover

What is the Goal?

- Bridge the gap between computational thinking and formal math
- Discernable Linear Algebra in LEAN

How to Read a Lean Proof:

Provable proposition

Implicit argument

What to prove

Follow the stars to see the state of the proof at each step

```
theorem my_zero_add {n : Nat}: 0 + n = n := by ★
  induction n with
  | zero => (rw[Nat.add_zero])
  | succ n' ih => (rw[Nat.add_succ, ih])★
```

The respective LEAN InfoView states:

★▼Tactic state	★▼Tactic state	★▼Tactic state
1 goal	1 goal	No goals
n : Nat	0 + 0 = 0	
0 + n = n		
	★▼Tactic state	
	1 goal	
	n' : Nat	
	ih : 0 + n' = n'	
	0 + (n' + 1) = n' + 1	

Proof Method Induction (tactic)

- Base case “zero”
- Next case “succ n' ih”

Using tactics/theorems for the cases

Linear Algebra in Lean

The Task: Invertible Matrix Theorem



What slowed us down?

- Lack of matrix tooling in mathlib:

```
-- Distributes matrix · vector multiplication over a sum of vectors:
@[simp]
theorem mulVec_distr {i : N}
  (L : List (Fin i → C))
  (M : Matrix (Fin i) (Fin i) C)
  :
  M.mulVec L.sum = (L.map M.mulVec).sum :=
-- Distributes a dot product across a sum:
theorem dotProduct_distr {i : N}
  (v : (Fin i → C))
  (V : List (Fin i → C))
  :
  dotProduct v V.sum = (V.map (dotProduct v)).sum :=
```

- Unfamiliarity with LEAN:

Wrong!

```
def orthogComplement {i : N}
  (Sv1 : Submodule C (Fin i → C))
  (Sv2 : Submodule C (Fin i → C))
  :=
  (∀ v ∈ Sv1, ∀ u ∈ Sv2, dotProduct u v = 0)
```

Notable Lemmas:

```
-- Allows distributing a vector multiply over a sum of vectors
multiplied by a coefficient.
-- Useful when translating from a basis.
theorem finset_scalar_mul_ignore {i : N}
  (f : (Fin i → C) → C)
  (V : Finset (Fin i → C))
  :
  ∃ V2 : List (Fin i → C),
  V2.sum = ∑ v ∈ V, (f v) · v ∧ ∀ v ∈ V2, ∃ v1 ∈ V, v = (f v1)·v1
-- The span of a nullspace is the same as the nullspace itself.
@[simp]
theorem nsSpan_eq_ns {i : N}
  (M : Matrix (Fin i) (Fin i) C)
  :
  nullspaceSpan M = nullspace M :=
```

- Study the most general representation of your theorem first. Mathlib rewards working foundationally.
- Don't get attached to representations! Restarting is often faster than fixing.

Transcribing Discrete Relations in LEAN

To add to the recourses students can use when learning Discrete Math and LEAN.

Here are examples from Section 9.1 of the Textbook

LEAN, Mathlib, & Tactics

Example proving reflexivity on a discrete relation

Defining the terms we need for the proof

```
universe u
def Refl {A : Type u} (r : A → A → Prop) :=
  ∀ x : A, r x x
abbrev Relation (A : Type u) (B : Type u) :=
  A → B → Prop
def R3 (x y : Fin 4) := match x with
| 0 => y = 0 ∨ y = 1 ∨ y = 3
| 1 => y = 0 ∨ y = 1
| 2 => y = 2
| 3 => y = 0 ∨ y = 3
```

The LEAN proof for saying the relation R3 is reflexive

```
example: Refl R3 := by
  simp [Refl, R3]
  intro x
  match x with
  | 0 => exact Or.inl rfl
  | 1 => exact Or.inr rfl
  | 2 => rfl
  | 3 => exact Or.inr rfl
```

How powerful are tactics?

The usage of Mathlib's Fin.cases tactic and LEAN's simp tactic completes many logical proof steps for us.

Without these tactics the process becomes more verbose

With utilization:

```
abbrev Symm {A : Type u} (r : Relation A A) :=
  ∀ x y : A, r x y → r y x
def R2 (x y : Fin 4) := match x with
| 0 => y = 0 ∨ y = 1
| 1 => y = 0
| _ => false
```

Without utilization:

```
theorem h_20 :
  (2 : Fin 4) = (0 : Fin 4) → False := by
  intro h
  apply Fin.noConfusion h
  intro h2
  rw[Nat.zero_mod] at h2
  apply Nat.noConfusion h2
theorem h_24 : 2 % 4 = 2 := by
  rfl
theorem h_21 :
  (2 : Fin 4) = (1 : Fin 4) → False := by
  intro h
  apply Fin.noConfusion h
  intro h2
  rw[Nat.one_mod] at h2
  rw[h_24] at h2
  apply Nat.succ_ne_self at h2
  exact h2
```

```
import Mathlib.Tactic.FinCases
example: Symm R2 := by
  simp [Symm, R2]
  intro x y
  fin_cases x <|>
  fin_cases y <|>
  simp
```

```
example: Symm R2 := by
  simp [Symm, R2]
  intro x y h
  match x with
  | 0 =>
    match y with
    | 0 => exact h
    | 1 => exact rfl
    | 2 => exact
      (Or.elim h
        (λ h20 => h_20 h20)
        (λ h21 => h_21 h21)
      )
  | 3 => simp at h
  | 1 =>
    match y with
    | 0 => exact (Or.inr rfl)
    | 1 => exact (h)
    | 2 => simp at h
    | 3 => simp at h
  | 2 => simp at h
  | 3 => simp at h
```

More examples in the respective GitHub repository.